

REGEX in MySQL

Short instructional guide

A regular expression, usually called REGEX, is a pattern used to search text. In MySQL, REGEX is useful when you need more flexible pattern matching than LIKE can provide. Instead of searching for only simple wildcards, you can search for letters, digits, repeated characters, starting patterns, ending patterns, and more complex text rules.

Why use REGEX?

- Search for values that begin with certain letters.
- Check whether a string contains only digits or only letters.
- Find data that follows a pattern such as phone numbers, codes, or email-like text.
- Replace part of a string when it matches a pattern.

Basic REGEX matching

The most common way to test a pattern in MySQL is with REGEXP_LIKE(). Older examples may also use REGEXP or RLIKE.

```
SELECT column_name
FROM table_name
WHERE REGEXP_LIKE(column_name, 'pattern');
```

Common pattern symbols

- ^ = start of the string
- \$ = end of the string
- . = any single character
- [ABC] = one character from the set shown
- [0-9] = any digit from 0 through 9
- + = one or more of the previous item
- * = zero or more of the previous item

Example 1: Names that start with A

```
SELECT first_name
FROM customer
WHERE REGEXP_LIKE(first_name, '^A');
```

The pattern ^A means the value must start with the letter A. This is useful when you want to find names such as Amy, Alice, or Aaron.

Example 2: Values made of digits only

```
SELECT phone
FROM customer
WHERE REGEXP_LIKE(phone, '^[0-9]+$');
```

This pattern checks that the value starts with a digit, ends with a digit, and contains one or more digits in between. Values with letters, spaces, or symbols will not match.

Example 3: Codes with a specific format

```
SELECT product_code
FROM inventory
WHERE REGEXP_LIKE(product_code, '^[A-Z]{3}-[0-9]{4}$');
```

This example looks for codes such as ABC-1234. It expects exactly three uppercase letters, one hyphen, and four digits.

REGEX functions

MySQL includes additional functions for extracting, locating, and replacing text with regular expressions.

REGEXP_SUBSTR()

```
SELECT REGEXP_SUBSTR('Order #A-1045 shipped', '[A-Z]-[0-9]{4}') AS result;
```

This returns the part of the string that matches the pattern. The result would be A-1045.

REGEXP_REPLACE()

```
SELECT REGEXP_REPLACE('abc-123', '[0-9]+', '###') AS result;
```

This replaces the digits with ###. The returned value would look like abc-###.

REGEXP_INSTR()

```
SELECT REGEXP_INSTR('Room 204B', '[0-9]+') AS position;
```

This returns the starting position of the first match in the string.

LIKE vs REGEX

- LIKE is easier for simple searches such as values that start with A or end with .com.
- REGEX is better when you need more control, such as fixed formats, repeated characters, or groups of allowed characters.
- Use REGEX when the search rule is more complex than a simple wildcard pattern.

Best practices

- Test your pattern on a few sample values before using it in a large query.
- Keep patterns readable. A shorter clear pattern is usually better than a very long one.
- Use ^ and \$ when you want the whole value to match the pattern, not just part of it.
- Be careful with special characters because they may have a special regex meaning.

Practice queries

```
-- 1. Find emails ending in .edu
SELECT email
FROM student
WHERE REGEXP_LIKE(email, '\.edu$');
```

```
-- 2. Find last names that contain only letters
SELECT lname
FROM student
WHERE REGEXP_LIKE(lname, '^[A-Za-z]+$');
```

```
-- 3. Find ZIP codes in 5-digit format
SELECT zip
FROM address
WHERE REGEXP_LIKE(zip, '^([0-9]){5}$');
```

Summary

REGEX gives MySQL users a powerful way to search text patterns. It is especially useful for validation, filtering, cleanup, and extraction tasks. Once you understand a few basic pattern symbols, you can write queries that search text much more precisely than LIKE.